CSA0613 - Design and Analysis of Algorithms

CO3-AT2-Case Study Based Assessment

Divyesh S P (192424147)

Online Gaming Leaderboard Using Quick Sort Algorithm

Aim

To implement the Quick Sort algorithm for sorting player scores in an online gaming leaderboard and analyze its efficiency, pivot selection strategies, worst-case performance, and optimization using randomized pivot selection.

Problem Statement

Online gaming platforms continuously update player scores. Whenever a player finishes a game, the leaderboard must be reordered quickly. An efficient sorting algorithm is required to display rankings in real time. Quick Sort is chosen because of its average-case time complexity of **O(n log n)** and fast practical performance.

Objectives

- Implement Quick Sort.
- Sort player scores in descending order.
- Analyze Divide and Conquer approach.
- Study pivot selection.
- Compare best, average and worst case.
- Implement Randomized Quick Sort.
- Measure execution time.

Software Requirements

- Python 3.x
- VS Code / Google Colab
- time module
- random module

Algorithm

Step 1

Store player names and scores.

Step 2

Choose a pivot.

Step 3

Partition the scores.

Step 4

Recursively sort left and right partitions.

Step 5

Display leaderboard.

6. Divide and Conquer Explanation

Divide

Choose one player score as pivot.

↓

Conquer

Move lower scores to one side and higher scores to another.

↓

Combine

Recursively sort both parts.

↓

Final Sorted Leaderboard

7.

Flowchart

Start

↓

Input Player Scores

↓

Choose Pivot

↓

Partition Scores

↓

Quick Sort Left

↓

Quick Sort Right

↓

Merge Result

↓

Display Leaderboard

↓

End

Python Implementation

```
import random
import time

players = [
    ("Alex", 2500),
    ("John", 3200),
    ("David", 1800),
    ("Emma", 4100),
    ("Sophia", 2900),
    ("Liam", 3600),
    ("Noah", 2100),
    ("Olivia", 3900)
]

def partition(arr, low, high):
    pivot = arr[high][1]
    i = low - 1

    for j in range(low, high):
        if arr[j][1] >= pivot:   # Descending order
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def randomized_partition(arr, low, high):
    random_index = random.randint(low, high)
    arr[random_index], arr[high] = arr[high], arr[random_index]
    return partition(arr, low, high)

def randomized_quick_sort(arr, low, high):
    if low < high:
        pi = randomized_partition(arr, low, high)
        randomized_quick_sort(arr, low, pi - 1)
        randomized_quick_sort(arr, pi + 1, high)

# Execute
start = time.time()

randomized_quick_sort(players, 0, len(players) - 1)

end = time.time()

print("===== ONLINE GAMING LEADERBOARD =====\n")

for i, player in enumerate(players, start=1):
    print(f"{i}. {player[0]} - {player[1]}")

print("\nExecution Time:", end - start)
```

Output

```
••• ===== ONLINE GAMING LEADERBOARD =====

    1. Emma - 4100
    2. Olivia - 3900
    3. Liam - 3600
    4. John - 3200
    5. Sophia - 2900
    6. Alex - 2500
    7. Noah - 2100
    8. David - 1800

    Execution Time: 0.00020551681518554688
```

Time Complexity Analysis

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$
Space Complexity	O(log n)

Pivot Selection Analysis

First Element

- Very simple.
- Poor for already sorted data.

Last Element

- Easy to implement.
- Can lead to worst-case performance.

Middle Element

- Better balance than first/last.

Random Pivot (Recommended)

- Randomly selects a pivot.
- Reduces the probability of worst-case partitions.
- Best choice for real-time gaming leaderboards.

Advantages

- Fast average performance.
- Efficient for large datasets.
- In-place sorting.
- Suitable for dynamic leaderboards.

Limitations

- Worst-case complexity is $O(n^2)$.
- Not a stable sorting algorithm.
- Performance depends on pivot selection.

Applications

- Online gaming leaderboards.
- Sports rankings.
- E-commerce product ranking.
- Student ranking systems.
- Stock market analysis.

Result

The Quick Sort algorithm successfully sorted player scores in descending order to generate a real-time gaming leaderboard. Using randomized pivot selection improved the robustness of the algorithm by reducing the likelihood of worst-case performance, making it well suited for dynamic leaderboard applications.